

CDT 6 - LECTURE SUMMARY

CDT6 Topics Covered	Divide and Conquer General method
Motivation (Why you (students) should learn these topics?)	(As course faculty, how do you motivate your course students to learn these topics? Students will learn the effective analysis of Divide and conquer technique , Divide and Conquer is an algorithmic pattern. In algorithmic methods, the design is to take a dispute on a huge input, break the input into minor pieces, decide the problem on each of the small pieces, and then merge the piecewise solutions into a global solution
Lecture Learning Outcomes (LLOs): After completion of this lecture, you should be able to...	
LLO1 On topic 1	recurrence $T(n) = 2 T(n/2) + n$ using recursive tree method
LLO2 On topic 1	Solve recurrence using recursive tree method $T(n) = T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + T\left(\left\lfloor \frac{2n}{3} \right\rfloor\right) + n$

CDT 6 – Lecture Summary – Key Takeaways

Divide and Conquer is an algorithmic paradigm (sometimes mistakenly called "Divide and Concur" - a funny and apt name), similar to Greedy and Dynamic Programming. A typical Divide and Conquer algorithm solves a problem using the following three steps.

1. **Divide:** Break the given problem into subproblems of same type. This step involves breaking the problem into smaller sub-problems. Sub-problems should represent a part of the original problem. This step generally takes a recursive approach to divide the problem until no sub-problem is further divisible. At this stage, sub-problems become atomic in nature but still represent some part of the actual problem.
2. **Conquer:** Recursively solve these sub-problems. This step receives a lot of smaller sub-problems to be solved. Generally, at this level, the problems are considered 'solved' on their own.
3. **Combine:** Appropriately combine the answers. When the smaller sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem. This algorithmic approach works recursively and conquer & merge steps works so close that they appear as one.
This method usually allows us to reduce the time complexity by a large extent.

For example, Bubble Sort uses a complexity of $O(n^2)$, whereas quicksort (an application Of Divide And Conquer) reduces the time complexity to $O(n \log(n))$. Linear Search has time complexity $O(n)$, whereas Binary Search (an application Of Divide And Conquer) reduces time complexity to $O(\log(n))$.

- A Divide and Conquer Algorithm to find a key in an array:

CDT 6 - LECTURE SUMMARY

```
-- Precondition: S is a sorted list
index binsearch(number n, index low, index high,
    const keytype S[], keytype x)
    if low ≤ high then
        mid = (low + high) / 2
        if x = S[mid] then
            return mid
        elseif x < S[mid] then
            return binsearch(n, low, mid-1, S, x)
        else
            return binsearch(n, mid+1, high, S, x)
    else
        return 0
end binsearch
```

- Divide: search lower or upper half
 - Divide: select lower or upper half
 - Conquer: search selected half
 - Combine: None

Many algorithms are recursive in nature. When we analyze them, we get a recurrence relation for time complexity. We get running time on an input of size n as a function of n and the running time on inputs of smaller sizes. For example in [Merge Sort](#), to sort a given array, we divide it in two halves and recursively repeat the process for the two halves. Finally we merge the results. Time complexity of Merge Sort can be written as $T(n) = 2T(n/2) + cn$.

There are many other algorithms like Binary Search, Tower of Hanoi, etc.

There are mainly three ways for solving recurrences.

1) Substitution Method: We make a guess for the solution and then we use mathematical induction to prove the guess is correct or incorrect.

For example consider the recurrence $T(n) = 2T(n/2) + n$

We guess the solution as $T(n) = O(n \log n)$. Now we use induction to prove our guess.

We need to prove that $T(n) \leq cn \log n$. We can assume that it is true for values smaller than n .

$$\begin{aligned}
 T(n) &= 2T(n/2) + n \\
 &\leq 2cn/2 \log(n/2) + n \\
 &= cn \log n - cn \log 2 + n \\
 &= cn \log n - cn + n \\
 &\leq cn \log n
 \end{aligned}$$

2) Recurrence Tree Method: In this method, we draw a recurrence tree and calculate the time taken by every level of tree. Finally, we sum the work done at all levels. To draw the recurrence tree, we start from the given recurrence and keep drawing till we find a pattern among levels.

CDT 6 - LECTURE SUMMARY

The pattern is typically a arithmetic or geometric series.

For example consider the recurrence relation

$$T(n) = T(n/4) + T(n/2) + cn^2$$

$$\begin{array}{c} cn^2 \\ / \quad \backslash \\ T(n/4) \quad T(n/2) \end{array}$$

If we further break down the expression $T(n/4)$ and $T(n/2)$, we get following recursion tree.

$$\begin{array}{c} cn^2 \\ / \quad \backslash \\ c(n^2)/16 \quad c(n^2)/4 \\ / \quad \backslash \quad / \quad \backslash \\ T(n/16) \quad T(n/8) \quad T(n/8) \quad T(n/4) \end{array}$$

Breaking down further gives us following

$$\begin{array}{c} cn^2 \\ / \quad \backslash \\ c(n^2)/16 \quad c(n^2)/4 \\ / \quad \backslash \quad / \quad \backslash \\ c(n^2)/256 \quad c(n^2)/64 \quad c(n^2)/64 \quad c(n^2)/16 \\ / \quad \backslash \quad / \quad \backslash \quad / \quad \backslash \quad / \quad \backslash \end{array}$$

To know the value of $T(n)$, we need to calculate sum of tree nodes level by level. If we sum the above tree level by level, we get the following series

$$T(n) = c(n^2 + 5(n^2)/16 + 25(n^2)/256) + \dots$$

The above series is geometrical progression with ratio $5/16$.

To get an upper bound, we can sum the infinite series.

We get the sum as $(n^2)/(1 - 5/16)$ which is $O(n^2)$

CDT6. LECTURE LEVEL PRACTICE PROBLEMS (LLPs) to test the LLOs

To test whether you achieved the learning outcomes of this lecture, you should be able to solve the following LLPs, in the class itself, after completion of the lecture. Minimum one question / problem (LLP) is designed to test each expected LLO.

- LLP1 Illustrate Divide and conquer general method
- Solve recurrence $T(n) = 2 T(n/2) + n$ using recursive tree method
-
- LLP2 Demonstrate and solve few recurrence relations through divide and conquer.
- Solve recurrence using recursive tree method

$$T(n) = T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + T\left(\left\lfloor \frac{2n}{3} \right\rfloor\right) + n$$